

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: String , I/O, Formatting, Regex, Serializable, Console

Total Questions : 57

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: String,Files,IO,Date and Serializable

Questions Question - 1

Which of the following return true?

- 1."das" == new String("das")
- 2."das".equals("das")
- 3."das".equals(new Button("das"))
- 4.None of the above

Explanation :

A is the correct answer.

== compare the addresses and equals() methods checks for content. In the "das" == new String("das") , new String("das") point to different address then "das".

Question - 2

Which of the following is correct?

- 1.String temp [] = new String {"j" "a" "z"};
- 2.String temp [] = { "j " " b" "c"};
- 3.String temp = {"a", "b", "c"};
- 4.String temp [] = {"a", "b", "c"};

Explanation :

D is the correct answer.

String temp [] = {"a", "b", "c"}; where temp[] is String array.

Question - 3

What is the output?

```
public class Test {  
  
    public static void main(String[] args) {  
  
        StringBuffer sb = new StringBuffer("ssss");  
    }  
}
```

```

        StringBuffer sb_2 = new StringBuffer("ssss");
        System.out.println("sb equals sb_2 : " + sb.equals(sb_2));

    }

}

```

- 1.sb equals sb_2 : false
- 2.sb equals sb_2 : true
- 3.Can't say
- 4.None of the above

Explanation :

A is the correct answer.

StringBuffer class DOES NOT override the equals() method. Therefore, it uses Object class' equals(), which only checks for equality of the object references

Question - 4

What is the output?

```

public class Test {

    public static void main(String[] args) {

        StringBuffer sb = new StringBuffer("ssss");
        String st = new String("ssss");
        System.out.println("st equals sb : " + st.equals(sb));

    }

}

```

- 1.st equals sb : false
- 2.st equals sb : true
- 3.can't say
- 4.None of the above

Explanation :

A is the correct answer.

String's equals() method checks if the argument is of type string, if not it returns false

Question - 5

What is true about StringBuilder ?

- 1.StringBuilder is a drop-in replacement for StringBuffer in cases where thread safety is not an issue.
- 2.StringBuilder is NOT synchronized
- 3.StringBuilder offers FASTER performance than StringBuffer
- 4.All of the above

Explanation :

D is the correct answer.

J2SE5.0 added the StringBuilder class, which is a drop-in replacement for StringBuffer in cases where thread safety is not an issue. Because StringBuilder is NOT synchronized, it offers FASTER performance than StringBuffer.

Question - 6

What is the output ?

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Test {

    public static void main(String... args) {

        Pattern p = Pattern.compile("a*b");
        Matcher m = p.matcher("aaaaab");
        boolean b = m.matches();
        System.out.println(b);

    }
}
```

- 1.true
- 2.false
- 3.compile error
- 4.None of the error

Explanation :

A is the correct answer.

a*b means a zero or more time and b should be present.

Question - 7

What is the output ?

```

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Test {

    public static void main(String... args) {

        Pattern p = Pattern.compile("a*b");
        Matcher m = p.matcher("b");
        boolean b = m.matches();
        System.out.println(b);

    }
}

```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

a*b means a zero or more time and b should be present.

Question - 8

What is the output ?

```

public class Test {

    public static void main(String... args) {

        Pattern p = Pattern.compile("a*b");
        Matcher m = p.matcher("aaaa");
        boolean b = m.matches();
        System.out.println(b);

    }
}

```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

B is the correct answer.

a*b means a zero or more times and b should be there.

Question - 9

What is output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a+b?");  
        Matcher m = p.matcher("aaaa");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

a+ means a , one or more times b? means b , once or not at all

Question - 10

What is output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a+b?");  
        Matcher m = p.matcher("aaaabb");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

B is the correct answer.

a+ means a , one or more times b? means b , once or not at all

Question - 11

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a+b?");  
        Matcher m = p.matcher("b");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

B is the correct answer.

a+ means a , one or more times b? means b , once or not at all

Question - 12

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a+b?c*");  
        Matcher m = p.matcher("ab");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true

- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

X? X, once or not at all X* X, zero or more times X+ X, one or more times

Question - 13

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a{3}b?c*");  
        Matcher m = p.matcher("aaab");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true
- 2.false
- 3.Compile Error
- 4.None of the above

Explanation :

A is the correct answer.

X? X, once or not at all X* X, zero or more times X+ X, one or more times X{n} X, exactly n times X{n,} X, at least n times X{n,m} X, at least n but not more than m times

Question - 14

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a{3,}b?c*");  
        Matcher m = p.matcher("aab");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

```
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

B is the correct answer.

X? X, once or not at all X* X, zero or more times X+ X, one or more times X{n} X, exactly n times X{n,} X, at least n times X{n,m} X, at least n but not more than m times

Question - 15

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a{1,3}b?c*");  
        Matcher m = p.matcher("aaab");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

X? X, once or not at all X* X, zero or more times X+ X, one or more times X{n} X, exactly n times X{n,} X, at least n times X{n,m} X, at least n but not more than m times

Question - 16

What is the output ?


```

public class Test {

    public static void main(String... args) {

        Pattern p = Pattern.compile("a{1,3}b?c*");
        Matcher m = p.matcher("aaaab");
        boolean b = m.matches();
        System.out.println(b);

    }
}

```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

B is the correct answer.

X? X, once or not at all X* X, zero or more times X+ X, one or more times X{n} X, exactly n times X{n,} X, at least n times X{n,m} X, at least n but not more than m times

Question - 17

What is the output ?

```

public class Test {

    public static void main(String... args) {

        Pattern p = Pattern.compile("[,\\s]+");

        String[] result = p.split("one,two, three four , five");
        for (int i=0; i<result.length; i++) {
            System.out.println( result[i] );
        }

    }
}

```

- 1.one two three four five
- 2.one two three five
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

[,\s]+ means comma or white space one or more time so split based on the criteria.

Question - 18

What is the output ?

```
public class Test {  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("[\\s]+");  
  
        String[] result = p.split("one,two, three    four,  five");  
        for (int i=0; i<result.length; i++) {  
            System.out.println( result[i] );  
        }  
    }  
}
```

- 1.one,two, three four, five
- 2.one, two three four, five
- 3.one,two, three five
- 4.None of the above

Explanation :

A is the correct answer.

[\\s]+ means white space one or more time so split based on the criteria.

Question - 19

"Instances of Pattern class are immutable and are safe for use by multiple concurrent threads.

Instances of the Matcher class are not safe for such use. "

Is the above statement true ?

- 1.true
- 2.false
- 3.Can't say
- 4.None of the above

Explanation :

A is the correct answer.

Instances of Pattern class are immutable and are safe for use by multiple concurrent threads. Instances of the Matcher class are not safe for such use.

Question - 20

`str.split (" ");` is equal to `str.split ("\\s")`
Is the above statement true ?

- 1.true
- 2.false
- 3.can't say
- 4.none of the above

Explanation :

A is the correct answer.

\\s is white space

Question - 21

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        String str = "This is a string object";  
        String[] words = str.split (" ");  
        for (String word : words) {  
            System.out.println (word);  
        }  
    }  
}
```

- 1.This is a string object
- 2.There no split method in String class
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

The split() method takes a parameter giving the regular expression to use as a delimiter and returns a String array containing the tokens so delimited. Using split() function:

Question - 22

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        String input = "1 fish 2 fish red fish blue fish";  
        Scanner s = new Scanner(input).useDelimiter("\\s*fish\\s*");  
        System.out.println(s.nextInt());  
        System.out.println(s.nextInt());  
        System.out.println(s.next());  
        System.out.println(s.next());  
        s.close();  
  
    }  
}
```

1. There is no object name Scanner so Compile error
2. 1 2 red blue
3. Runtime Exception
4. None of the above

Explanation :

B is the correct answer.

java.util.Scanner is a simple text scanner which can parse primitive types and strings using regular expressions

Question - 23

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        String str = "Mydfresdhgtjjjsdjh";  
        String[] words = str.split ("d");  
        for (String word : words) {  
            System.out.println (word);  
        }  
  
    }  
}
```

1. My fres hhgtjjs jh

- 2. My fres hgtjjs
- 3. Compile error
- 4. None of the above

Explanation :

A is the correct answer.

The split() method takes a parameter giving the regular expression to use as a delimiter and returns a String array containing the tokens so delimited. Using split() function

Question - 24

Which statement is true about "java.io.File";

- 1. Files and directories are accessed and manipulated via the java.io.File class
- 2. Files accessed and manipulated via the java.io.File class not directories
- 3. both are true
- 4. None of the above.

Explanation :

A is the correct answer.

Files and directories are accessed and manipulated via the java.io.File class. The File class does not actually provide for input and output to files. It simply provides an identifier of files and directories.

Question - 25

Which statement is true?

- 1. FileReader is meant for reading streams of characters
- 2. FileInputStream is meant for reading streams of raw bytes
- 3. FileReader is meant for reading streams of characters and raw bytes both
- 4. None of the above

Explanation :

A and B is the correct answer.

FileReader is meant for reading streams of characters. For reading streams of raw bytes, consider using a FileInputStream.

Question - 26

Which statement is true?

1. FileWriter is meant for writing streams of characters
2. FileOutputStream is meant for writing streams of raw bytes
3. FileReader is meant for writing streams of characters and raw bytes both
4. None of the above

Explanation :

A and B is the correct answer.

FileReader is meant for reading streams of characters. For reading streams of raw bytes, consider using a FileInputStream.

Question - 27

Is the bellow statement is true?

"Only objects that support the java.io.Serializable or java.io.Externalizable interface can be read from streams"

1. true
2. false
3. can't say
4. none of the above

Explanation :

A is the correct answer.

Only objects that support the java.io.Serializable or java.io.Externalizable interface can be read from streams

Question - 28

What is the output for the below code?

```
public class A {
    public A() {
        System.out.println("A");
    }
}

public class B extends A implements Serializable {
    public B() {
        System.out.println("B");
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        B b = new B();
    }
}
```

```

        ObjectOutputStream save = new ObjectOutputStream(new
FileOutputStream("datafile"));
        save.writeObject(b);
        save.flush();

        ObjectInputStream restore = new ObjectInputStream(new
FileInputStream("datafile"));
        B z = (B) restore.readObject();

    }
}

```

- 1.A B A
- 2.A B A B
- 3.B B
- 4.B

Explanation :
A is the correct answer.

On the time of deserialization , the Serializable object not create new object. So constructor of class B does not called. A is not Serializable object so constructor is called.

Question - 29

What is the output for the below code?

```

public class A {
    public A() {
        System.out.println("A");
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        A a = new A();

        ObjectOutputStream save = new ObjectOutputStream(new
FileOutputStream("datafile"));
        save.writeObject(a);
        save.flush();

        ObjectInputStream restore = new ObjectInputStream(new
FileInputStream("datafile"));
        A z = (A) restore.readObject();
    }
}

```

```
    }  
}
```

- 1.A A
- 2.A
- 3.java.io.NotSerializableException
- 4.None of the above

Explanation :

C is the correct answer.

Class A does not implements Serializable interface. So throws NotSerializableException on trying to Serialize a non Serializable object.

Question - 30

What is the output for the below code?

```
public class A implements Serializable{  
    public A() {  
        System.out.println("A");  
    }  
}  
  
public class Test {  
  
    public static void main(String... args) throws Exception {  
        A a = new A();  
  
        ObjectOutputStream save = new ObjectOutputStream(new  
FileOutputStream("datafile"));  
        save.writeObject(a);  
        save.flush();  
  
        ObjectInputStream restore = new ObjectInputStream(new  
FileInputStream("datafile"));  
        A z = (A) restore.readObject();  
  
    }  
}
```

- 1.A A
- 2.A
- 3.Runtime Exception
- 4.Compile with error

Explanation :

B is the correct answer.

On the time of deserialization , the Serializable object not create new object. So constructor of class A does not called.

Question - 31

Can static variables are Serialized ?

- 1.yes
- 2.No,static can't be Serialized.
- 3.may or may not Serialized
- 4.None of the above

Explanation :

B is the correct answer.

No,static and transient can't be Serialized.

Question - 32

Which statement is true?

- 1.Implementing the Serializable interface allows object serialization to save and restore the entire state of the object
- 2.Implementing the Serializable interface allows object serialization to save state of the object and can't restore the entire state
- 3.Both are true
- 4.None of the above

Explanation :

A is the correct answer.

Implementing the Serializable interface allows object serialization to save and restore the entire state of the object

Question - 33

Which statement is true?

- 1.static and transient fields are NOT serialized
- 2.static and transient fields are can be serialized
- 3.Both are true
- 4.None of the above

Explanation :

A is the correct answer.

static and transient fields are NOT serialized

Question - 34

```
public class A {}

public class B implements Serializable {
    A a = new A();
    public static void main(String... args){
        B b = new B();
        try{
            FileOutputStream fs = new FileOutputStream("b.ser");
            ObjectOutputStream os = new ObjectOutputStream(fs);
            os.writeObject(b);
            os.close();

        }catch(Exception e){
            e.printStackTrace();
        }

    }
}
```

What is the output for the above code ?

- 1.Compilation Fail
- 2.java.io.NotSerializableException: Because class A is not Serializable.
- 3.No Exception
- 4.None of the above

Explanation :

B is the correct answer.

java.io.NotSerializableException:A Because class A is not Serializable.

Question - 35

```
public class A {
    public A() {
        System.out.println("A");
    }
}

public class B extends A implements Serializable {
    public B() {
        System.out.println("B");
    }
}

public class Test {
```

```

        public static void main(String... args) throws Exception {
            B b = new B();

            ObjectOutputStream save = new ObjectOutputStream(new
FileOutputStream("datafile"));
            save.writeObject(b);
            save.flush();

            ObjectInputStream restore = new ObjectInputStream(new
FileInputStream("datafile"));
            B z = (B) restore.readObject();

        }
    }
}

```

What is the output?

- 1.A B A
- 2.A B A B
- 3.B B
- 4.B

Explanation :

A is the correct answer.

On the time of deserialization , the Serializable object not create new object. So constructor of class B does not called. A is not Serializable object so constructor is called.

Question - 36

What is the output for the below code?

```

public class A {
    public A() {
        System.out.println("A");
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        A a = new A();

        ObjectOutputStream save = new ObjectOutputStream(new
FileOutputStream("datafile"));
        save.writeObject(a);
        save.flush();

        ObjectInputStream restore = new ObjectInputStream(new
FileInputStream("datafile"));
        A z = (A) restore.readObject();
    }
}

```

```
    }  
}
```

- 1.A A
- 2.A
- 3.java.io.NotSerializableException
- 4.None of the above

Explanation :
C is the correct answer.

Class A does not implements Serializable interface. So throws
NotSerializableException on trying to Serialize a non Serializable object.

Question - 37

What is the output for the below code?

```
public class A implements Serializable{  
    public A() {  
        System.out.println("A");  
    }  
}  
  
public class Test {  
  
    public static void main(String... args) throws Exception {  
        A a = new A();  
  
        ObjectOutputStream save = new ObjectOutputStream(new  
FileOutputStream("datafile"));  
        save.writeObject(a);  
        save.flush();  
  
        ObjectInputStream restore = new ObjectInputStream(new  
FileInputStream("datafile"));  
        A z = (A) restore.readObject();  
  
    }  
}
```

- 1.A A
- 2.A
- 3.Runtime Exception
- 4.Compile with error

Explanation :

B is the correct answer.

On the time of deserialization , the Serializable object not create new object. So constructor of class A does not called.

Question - 38

```
public class A implements Serializable {
    transient int a = 7;
    static int b = 9;
}

public class B implements Serializable {

    public static void main(String... args){
        A a = new A();
        try {
            ObjectOutputStream os = new ObjectOutputStream(
                new FileOutputStream("test.ser"));
            os.writeObject(a);
            os.close();
            System.out.print( ++ a.b + " ");

            ObjectInputStream is = new ObjectInputStream(new
FileInputStream("test.ser"));
            A s2 = (A)is.readObject();
            is.close();
            System.out.println(s2.a + " " + s2.b);
        } catch (Exception x)
        {
            x.printStackTrace();
        }

    }

}
```

What is the output?

1.9 0 9

2.9 7 9

3.Runtime Exception

4.Compile with error

Explanation :

A is the correct answer.

static and transient variables are not serialized when an object is serialized.

Question - 39

Which is the correct way of Instantiate BufferedWriter object?

1. `BufferedWriter b1 = new BufferedWriter(new File("file.txt"));`
2. `BufferedWriter b1 = new BufferedWriter(new FileWriter("file.txt"));`
3. Both are true
4. None of the above

Explanation :

B is the correct answer.

Constructor of BufferedWriter is `public BufferedWriter(Writer out) { }` so `BufferedWriter b1 = new BufferedWriter(new FileWriter("file.txt"));` is correct one.

Question - 40

What will be the result of compiling and run the following code:

```
public class Test {  
  
    public static void main(String... args) throws Exception {  
        Integer i = 34;  
        long l = 341;  
        if(i.equals(l)){  
            System.out.println(true);  
        }else{  
            System.out.println(false);  
        }  
    }  
}
```

1. true
2. false
3. Compile error
4. None of the above

Explanation :

B is the correct answer.

`equals()` method for the integer wrappers will only return true if the two primitive types and the two values are equal.

Question - 41

What will be the result of compiling and run the following code:

```
public class Test {  
  
    public static void main(String... args) throws Exception {
```

```

Integer i = 34;
int l = 34;
if(i.equals(l)){
    System.out.println(true);
}else{
    System.out.println(false);
}

}

}

```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

equals() method for the integer wrappers will only return true if the two primitive types and the two values are equal.

Question - 42

When comparing java.io.BufferedWriter and java.io.FileWriter, which capability exist as a method in only one of two ?

- 1.closing the stream
- 2.flushing the stream
- 3.writting to the stream
- 4.writting a line separator to the stream

Explanation :

D is the correct answer.

A newLine() method is provided in BufferedWriter which is not in FileWriter.

Question - 43

What will be the result of compiling and run the following code:

```

public class Test {

    public static void main(String... args) throws Exception {
        File file = new File("test.txt");
    }

}

```

- 1.create new actual file name as test.txt

- 2.no actual file will be created.
- 3.Compile error.
- 4.None of the above

Explanation :

B is the correct answer.

creating a new instance of the class File, you're not yet making an actual file, you're just creating a filename

Question - 44

What will be the result of compiling and run the following code:
`public class Test {`

```
    public static void main(String... args) throws Exception {  
        File file = new File("test.txt");  
        System.out.println(file.exists());  
        file.createNewFile();  
        System.out.println(file.exists());  
    }  
}
```

- 1.true true
- 2.false true
- 3.false true
- 4.None of the above

Explanation :

B is the correct answer.

creating a new instance of the class File, you're not yet making an actual file, you're just creating a filename. So file.exists() return false. createNewFile() method created an actual file.so file.exists() return true.

Question - 45

What will be the result of compiling and run the following code:
`public class Test {`

```
    public static void main(String... args) throws Exception {  
        File file = new File("test.txt");  
        System.out.println(file.exists());  
        FileWriter fw = new FileWriter(file);  
        System.out.println(file.exists());  
    }  
}
```


- 1.true true
- 2.false true
- 3.false true
- 4.None of the above

Explanation :

B is the correct answer.

creating a new instance of the class File, you're not yet making an actual file, you're just creating a filename. So file.exists() return false. FileWriter fw = new FileWriter(file) do three things: It created a FileWriter reference variable fw. It created a FileWriter object, and assigned it to fw. It created an actual empty file out on the disk. So file.exists() return true .

Question - 46

What is the way to create a actual file ?

- 1.Invoke the createNewFile() method on a File object.
- 2.Create a Reader or a Writer or a Stream.
- 3.Both are true
- 4.None of the above

Explanation :

C is the correct answer.

Whenever you create an instance of a FileReader, a FileWriter, a PrintWriter, a FileInputStream, or a FileOutputStream classes, you automatically create a file, unless one already exists.

Question - 47

What will be the result of compiling and run the following code:

```
public class Test {  
  
    public static void main(String... args) throws Exception {  
        File myDir = new File("test");  
        // myDir.mkdir();  
        File myFile = new File(  
            myDir, "test.txt");  
        myFile.createNewFile();  
    }  
}
```

- 1.create directory "test" and a file name as "test.txt" within the the directory.
- 2.java.io.IOException: No such file or directory
- 3.Compile with error

4.None of the above

Explanation :

B is the correct answer.

// myDir.mkdir(); is commented so no directory, thatswhy exception.

Question - 48

```
public class A {}

public class B implements Serializable {
    private transient A a = new A();
    public static void main(String... args){
        B b = new B();
        try{
            FileOutputStream fs = new FileOutputStream("b.ser");
            ObjectOutputStream os = new ObjectOutputStream(fs);
            os.writeObject(b);
            os.close();

        }catch(Exception e){
            e.printStackTrace();
        }

    }
}
```

What is the output for the above code ?

- 1.Compilation Fail
- 2.java.io.NotSerializableException: Because class A is not Serializable.
- 3.No Exception
- 4.None of the above

Explanation :

C is the correct answer.

No java.io.NotSerializableException, Because class A variable is transient. transient variables are not Serializable.

Question - 49

What is the output for the below code ?

```
public class A {}

public class B implements Serializable {
    private static A a = new A();
    public static void main(String... args){
```

```

        B b = new B();
        try{
            FileOutputStream fs = new FileOutputStream("b.ser");
            ObjectOutputStream os = new ObjectOutputStream(fs);
            os.writeObject(b);
            os.close();

        }catch(Exception e){
            e.printStackTrace();
        }

    }

}

```

- 1.Compilation Fail
- 2.java.io.NotSerializableException: Because class A is not Serializable.
- 3.No Exception at Runtime
- 4.None of the above

Explanation :
C is the correct answer.

No java.io.NotSerializableException, Because class A variable is static. static variables are not Serializable.

Question - 50

Which statement is true ?

- 1.serialization applies only to OBJECTS.
- 2.Static variables are NEVER saved as part of the object's state. So static variables are not Serializable.
- 3.Both are true
- 4.None of the above

Explanation :
C is the correct answer.

serialization applies only to OBJECTS. static variables are related to class not OBJECT. so static variables are not Serializable.

Question - 51

```

public class Test {

    public static void main(String... args) throws Exception {
        Calendar c = new Calendar();
        System.out.println(c.getTimeInMillis());
    }

}

```

```
}
```

What is the output for the above code ?

- 1.Returns this Calendar's time value in milliseconds.
- 2.Compile error : Cannot instantiate the type Calendar
- 3.Runtime Exception
- 4.None of the above

Explanation :

B is the correct answer.

Compile error : Cannot instantiate the type Calendar. In order to create a Calendar instance, you have to use one of the overloaded getInstance() static factory methods:
Calendar cal = Calendar.getInstance();

Question - 52

```
public class Test {  
  
    public static void main(String... args) throws Exception {  
        Date d = new Date(1123631685981L);  
        DateFormat df = new DateFormat();  
        System.out.println(df.format(d));  
    }  
}
```

What is the output for the above code ?

- 1.An exception is thrown at runtime.
- 2.1123631685981L
- 3.Compilation fails
- 4.None of the above

Explanation :

C is the correct answer.

DateFormat objects must be created using a static method such as DateFormat.getInstance() or DateFormat.getDateInstance().

Question - 53

Which statement is true?

- 1.The DateFormat.getDate() is used to convert a String to a Date instance
- 2.If a NumberFormat instance's Locale is to be different than the current Locale, it must be specified at creation time.
- 3.Both DateFormat and NumberFormat objects can be constructed to be Locale specific
- 4.2 and 3 is true

Explanation :

D is the correct answer.

DateFormat.parse() is used to convert a String to a Date.

Question - 54

Which statement is true about java.io.Console class ?

- 1.Console has readPassword() method that disables console echo and returns a char array
- 2.Console has readLine() method that disables console echo and returns a char array]
- 3.Both of the above statements are true
- 4.None of the above

Explanation :

A is the correct answer.

Console has readPassword() method that disables console echo and returns a char array. You can't see the password in console.

Question - 55

Which statement is true about java.io.Console class ?

- 1.you can't extend Console class because it is final
- 2.Console class is not final]
- 3.Console implements Flushable
- 4.None of the above

Explanation :

A and C is the correct answer.

public final class Console implements Flushable {}. you can't extend Console class because it is final.

Question - 56

What is the output?

```
import java.io.Console;

public class Test {
    public static void main(String... args) {

        Console con = System.console();
        boolean auth = false;

        if (con != null)
        {
            int count = 0;
            do
            {
                String uname = con.readLine(null);
                char[] pwd = con.readPassword("Enter %s's password: ",
uname);

                con.writer().write("\n\n");
            } while (!auth && ++count < 3);
        }
    }
}
```

- 1.NullPointerException
- 2.It works properly
- 3.Compile Error : No readPassword() method in Console class.
- 4.None of the above

Explanation :

A is the correct answer.

□passing a null argument to any method in Console class will cause a NullPointerException to be thrown.

Question - 57

How to get instance of java.io.Console class ?

- 1.Console con = System.console();
- 2.Console con = new Console();]
- 3.None of the above

Explanation :

A is the correct answer.

If this virtual machine has a console then it is represented by a unique instance of this class which can be obtained by invoking the System.console() method. If no console device is available then an invocation of that method will return null.

